Build the intelligence platform first, then build the executive experience on top of it.

The MVP should establish a reliable security data foundation, demonstrate meaningful risk intelligence, and deliver a working executive dashboard with a conversational AI assistant.

Cyber Command Center


Cyber Command Center
Enterprise Security Intelligence
Live • 11/08/2025 14:32:18 UTC
Last 24 Hours
John Doe
CIO
Executive Overview
Asset Intelligence
Vulnerability Management
Patch Compliance
Control Compliance (NIST 800-53)
Security Operations
Identity Security
Cloud Security
Network Security
Data Protection
Risk Management
AI Operations
Platform Health
Administration
Enterprise Risk Score
72 / 100
↓ 6% vs last week
Executive Cybersecurity Overview
Real-time insights into your enterprise security posture, risks and compliance.
Overall Posture: Healthy
Enterprise Risk Score
72 / 100
↓ 6%
Patch Compliance
94.2%
↑ 1.1%
Vulnerabilities
423
↓ 12%
NIST 800-53 Compliance
89.7%
↑ 0.5%
Security Posture Trend
1D 7D 30D 90D
Nov 2 Nov 3 Nov 4 Nov 5 Nov 6 Nov 7 Nov 8
Risk Score Compliance Patch Compliance
Top 5 Enterprise Risks
View All →
Risk
Severity
Business Impact
Status
Critical Prod Server Exposure
Critical
Production Disruption
Action Required
Unpatched Internet-Facing Application
High
Data Breach
In Progress
Excessive Privileged Access
High
Privilege Abuse
Action Required
Incomplete AC-2 Assessment
Medium
Compliance Gap
Planned
Cloud Storage Misconfiguration
Medium
Data Exposure
Monitoring
Key Security Metrics
View Details →
94.2%
Patch Compliance
↑ 1.1%
89.7%
NIST 800-53 Compliance
↑ 0.5%
412
Open Vulnerabilities
↓ 12%
8
Open Incidents
↓ 20%
Compliance by Framework
NIST 800-53 89.7%
ISO 27001 92.3%
TISAX 91.1%
0% 25% 50% 75% 100%
AI Security Assistant
What is the compliance level of PROD-SRV-001?
The compliance level for PROD-SRV-001 is 96% across NIST 800-53 controls.
Details:
NIST 800-53: 96%
ISO 27001: 94%
TISAX: 95%
Key Findings:
2 non-critical findings
Patch status: Up to date
Risk Level: Low
View asset details →
Last updated: Nov 8, 2025 14:30 UTC
Show similar assets
What are our top risks?
Patch overdue systems
Ask a question about your security...

Fig 1


Cyber Command Center
Enterprise Asset Intelligence
Total Assets
4,862
↑ 2%
Critical Assets
325
↑ 1%
Asset Risk Score
78 / 100
↓ 3%
Asset Distribution
4,862 Total
IT 68%
Cloud 18%
OT 14%
Assets by Business Unit
Manufacturing 32%
Engineering 28%
Corporate IT 22%
Supply Chain 18%
Asset Risk Heatmap
Critical 12 8 4
High 48 32 12
Medium 76 54 18
Low 40 30 10
IT Cloud OT
Top Risk Assets
Asset
Business Service
Risk Level
Key Issues
Action
PROD-SRV-001
Production Platform
Critical
3 Vulns, No Patch
Investigate
ENG-APP-102
Engineering Portal
High
1 Vuln, Exposed
Remediate
FIN-SRV-201
Finance System
High
Missing Patches
Remediate
Cyber Command Center
Asset 360° View
← Back to Assets
PROD-SRV-001
Production Application Server
Critical
Production
Manufacturing
Overview
Security Posture
Compliance
Risk History
Related Assets
Asset Profile
Asset ID: AST-10025
Hostname: PROD-SRV-001
IP Address: 10.20.15.45
Owner: Infrastructure Team
Business Unit: Manufacturing
Business Service: Production ERP
Criticality: Critical
Environment: Production
Location: Plant 1 – Floor 2
Security Posture
72 /100
Risk Score
3 Critical Vulns
1 High Vuln
Patch Status: Outdated
Control Status: Partial
Compliance Status
NIST 800-53 85%
ISO 27001 80%
TISAX 78%
Business Impact
Potential production disruption
(High Business Impact)
Recommended Actions
1 Patch critical vulnerabilities
2 Complete NIST control assessment
Cyber Command Center
Vulnerability Management
Critical 12
High 46
Total Vulnerabilities 234
Remediated (30d) 89%
Vulnerability Trend
Critical High Medium
Nov 2 Nov 3 Nov 4 Nov 5 Nov 6 Nov 7 Nov 8
Remediation Workflow
Vulnerability Identified (Tenable) – 11/08 10:24
Risk Prioritized (Critical)
Assigned to Team (Infra Team)
In Progress
Target: 11/10
Verification Pending
Top Vulnerabilities
CVE
Severity
Asset
Business Impact
Due Date
CVE-2025-1234
Critical
PROD-SRV-001
Production Impact
In Progress
CVE-2025-5678
High
ENG-APP-102
Engineering Delay
Assigned
CVE-2025-9012
High
FIN-SRV-201
Financial Impact
11/14/2025
Cyber Command Center
AI Security Assistant
What are our five biggest cybersecurity risks today?
Here are your top 5 cybersecurity risks today, based on current data:
1 Critical Production server exposure (PROD-SRV-001)
3 critical vulnerabilities + missing patch
2 High Excessive privileged access
248 users with admin rights
3 High Incomplete NIST control implementation
AC-2, IA-2, and others
4 Medium Cloud misconfiguration in GCP
Public storage bucket detected
5 Medium Data loss risk
Sensitive data in unprotected location
Would you like detailed information on any of these risks?
Ask a question about your security...
Workflow: Risk Investigation
Query Analyzed (2.1s)
Data Correlated (4 sources)
Insights Generated (3.4s)
Recommendations Ready
Sources
Tenable
Microsoft Defender
Google Chronicle
View Details →

Fig 2



Fig 3

Product Requirements Document (PRD) + Technical Architecture

Document version: 1.0
Product: Enterprise Cyber Command Center
Target users: CIO, CISO, CTO, Security Leadership, Infrastructure Leadership, Business Owners and Internal Audit
Deployment: On-premises first, GCP-enabled
Development approach: Claude Code-assisted engineering
Architecture: Multi-agent, event-driven, centrally orchestrated

---

PART 1. PRODUCT REQUIREMENTS DOCUMENT

1. Product objectives

The Cyber Command Center will consolidate enterprise cybersecurity information into one centralized intelligence platform.

It must provide:

1. Enterprise cybersecurity posture.
2. Enterprise asset intelligence.
3. Vulnerability and patch compliance.
4. NIST 800-53 control implementation and compliance.
5. Business-impact-based risk intelligence.
6. Security operations visibility.
7. Conversational AI for executive inquiries.
8. AI platform operational health and cost intelligence.

Business outcomes

Objective	Expected outcome
Centralized visibility	Eliminate fragmented cybersecurity reporting
Improved risk management	Identify the most important business risks
Compliance visibility	Establish continuous control monitoring
Asset intelligence	Understand security posture of every critical asset
Operational efficiency	Reduce manual security reporting
Executive decision support	Translate technical findings into business impact
AI adoption	Automate analysis and intelligence generation
Cost governance	Monitor AI usage, token consumption and inference expenses

2. Product personas

The platform will support different views based on user roles.

Persona	Primary requirements
CIO	Enterprise technology risk, resilience, investments and business impact
CISO	Security posture, threats, vulnerabilities and compliance
CTO	Application, cloud, infrastructure and engineering security
Security Operations	Incidents, detections and remediation
Infrastructure Leadership	Patch compliance, infrastructure risks and configuration
Application Owners	Application vulnerabilities and security controls
Business Owners	Risks affecting their business services
Internal Audit	Control implementation and evidence
Platform Administrator	Integrations, agents, models, usage and platform health

Implement RBAC and, where necessary, attribute-based access control.

---

3. MVP functional scope

I recommend eight initial functional modules.

Module	Priority	MVP
Executive Security Dashboard	P0	Yes
Enterprise Asset Intelligence	P0	Yes
Vulnerability Management	P0	Yes
Patch Compliance	P0	Yes
NIST Control Compliance	P0	Yes
Executive Risk Intelligence	P0	Yes
Conversational AI Assistant	P0	Yes
AI Operations and Token Monitoring	P0	Yes

Additional modules will follow.

Module	Target phase
Cloud Security	Phase 2
Identity Security	Phase 2
Network Security	Phase 2
Data Protection	Phase 2
Jira remediation integration	Phase 2
Business Resilience	Phase 2
Advanced OT Security	Phase 3
Application Security	Phase 3
Predictive Risk Analytics	Phase 4

Important: Although some integrations are deferred, the data model must accommodate their asset types and relationships from the beginning.

---

4. Detailed functional requirements

Module 1: Executive Security Dashboard

This is the default landing page.

Required KPIs

KPI	Description	Calculation
Enterprise Risk Score	Consolidated enterprise risk	Weighted risk model
Security Posture	Overall security health	Normalized security metrics
Patch Compliance	Percentage of compliant systems	Compliant / Applicable systems
Vulnerability Exposure	Critical and high-risk findings	Deduplicated vulnerability inventory
NIST Compliance	Control implementation status	Weighted applicable control assessment
Critical Assets	Number of critical business assets	Asset inventory
Open Incidents	Active security incidents	Chronicle/Defender
Remediation SLA	Findings remediated within SLA	SLA-compliant / Eligible findings

Every KPI must have:

- Current value.
- Previous-period value.
- Trend indicator.
- Data freshness.
- Source systems.
- Drill-down capability.

Executive risk presentation

The dashboard must show:

Top five enterprise cybersecurity risks.

Each risk should include:

- Risk title.
- Business unit.
- Affected business service.
- Affected assets.
- Risk rating.
- Business impact.
- Accountable owner.
- Recommended action.
- Last updated timestamp.

Do not simply rank vulnerabilities by CVSS.

Prioritization should consider exploitability, asset criticality, exposure, compensating controls, and business impact.

Module 2: Enterprise Asset Intelligence

This is the foundational service.

Asset categories

IT

- Endpoints.
- Servers.
- Applications.
- Databases.
- Network devices.
- Cloud resources.
- SaaS.
- Identities.

Cloud

- GCP projects.

- Compute instances.
- Storage.
- Kubernetes clusters.
- Containers.
- Service accounts.

OT placeholders

- PLC.
- HMI.
- SCADA.
- Industrial PCs.
- Robotics.
- MES.
- Production networks.
- IIoT.

OT assets will initially be maintained through authorized manual inventory updates.

Required asset information

Field	Required
Enterprise Asset ID	Yes
Hostname	Yes, where applicable
IP address	Yes, where applicable
MAC address	Yes, where applicable
Asset type	Yes
Asset owner	Yes
Business unit	Yes
Business service	Yes
Criticality	Yes
Location	Yes
Environment	Yes
Operating system	Where applicable
Security findings	Yes
Patch status	Where applicable
Control status	Yes
Risk rating	Yes

Asset intelligence functionality

The system must:

1. Import the existing asset database.
2. Collect asset information from integrated platforms.
3. Correlate records using multiple identifiers.
4. Detect duplicate records.

5. Identify unknown assets.
6. Identify assets without owners.
7. Identify assets without criticality.
8. Associate vulnerabilities with assets.
9. Associate controls with assets.
10. Maintain asset history.

An asset must not be automatically deleted because it disappears from one source.

It should instead be marked as potentially inactive until verified.

Module 3: Vulnerability Management

Dashboard metrics

- Total vulnerabilities.
- Critical vulnerabilities.
- High vulnerabilities.
- Exploitable vulnerabilities.
- Known exploited vulnerabilities.
- Overdue vulnerabilities.
- Vulnerability aging.
- Vulnerability distribution by business unit.
- Vulnerability distribution by asset criticality.

Required filters

- Severity.
- CVE.
- Asset.
- Business unit.
- Application.
- Environment.
- Remediation status.
- SLA status.
- Exploitability.

Risk prioritization

Use a risk-based prioritization model.

Suggested factors:

Factor	Consideration
CVSS	Technical severity
Exploitability	Whether exploitation is practical
Known exploitation	Threat intelligence
Internet exposure	External accessibility

Factor	Consideration
Asset criticality	Business importance
Business dependency	Operational consequences
Compensating controls	Existing protections
Vulnerability age	Exposure duration

The system should explain why a finding has a particular priority.

Module 4: Patch Compliance

Integrations:

- SCCM.
- Microsoft Intune.

Required metrics

- Enterprise patch compliance.
- OS patch compliance.
- Application patch compliance, where supported.
- Critical patch backlog.
- Systems exceeding patch SLA.
- Patch deployment failures.
- Systems with missing patches.
- Patch compliance by business unit.

Important calculation requirement

Do not calculate patch compliance by simply dividing patched assets by all inventoried assets.

The system must distinguish:

- Applicable systems.
- Non-applicable systems.
- Unknown patch status.
- Offline systems.
- Exempted systems.
- Systems with failed patch deployments.

Unknown must not be treated as compliant.

Module 5: NIST 800-53 Control Compliance

This will be our foundational GRC capability.

Control library

Start with NIST SP 800-53 Rev. 5.

Maintain mappings to:

- ISO 27001.
- TISAX / VDA ISA.

Control status

Use the following states:

1. Implemented.
2. Partially Implemented.
3. Not Implemented.
4. Not Applicable.
5. Not Assessed.
6. Evidence Pending.

Also maintain a separate effectiveness status.

An implemented control is not necessarily an effective control.

Control assessment workflow

Example: AC-2 Account Management.

```
NIST AC-2 Control
       |
       v
Identify Assessment Criteria
       |
       v
Collect Enterprise Evidence
       |
       +---- Entra ID
       +---- Active Directory
       +---- Access Reviews
       +---- Identity Policies
       |
       v
Run Deterministic Tests
       |
       v
AI-Assisted Evidence Analysis
       |
       v
Human Validation
       |
       v
Control Assessment
       |
       v
Evidence Repository
       |
       v
Executive Compliance Dashboard
```

Every control assessment must retain:

- Assessment date.
- Control owner.

- Assessment criteria.
- Evidence references.
- Test results.
- Assessor.
- Validation status.
- Exceptions.
- Remediation actions.

Critical requirement: The AI agent may recommend an assessment status, but it cannot independently declare a control compliant without satisfying the defined assessment and approval process.

Module 6: Executive Risk Intelligence

This is the intelligence engine that connects the other modules.

Example use case

A critical production server has:

- Three critical vulnerabilities.
- Missing security patches.
- Excessive privileged access.
- An incomplete AC-2 assessment.
- A dependency on a manufacturing application.

The Risk Intelligence service should produce one consolidated risk record.

Example:

Attribute	Result
Risk	Critical production server exposure
Severity	Critical
Business impact	Potential production disruption
Affected asset	PROD-SRV-001
Vulnerabilities	3 critical
Control gaps	AC-2, SI-2 assessment findings
Owner	Infrastructure
Recommended action	Prioritized remediation
Evidence	Linked source findings

The risk model should be explainable and version-controlled.

Module 7: Conversational AI Assistant

The chatbot should be accessible throughout the dashboard.

Required example interactions

Asset questions

- What is the compliance level of PROD-SRV-001?
- Which critical assets have no assigned owner?
- Show me assets with critical vulnerabilities and overdue patches.

Compliance questions

- What is our overall NIST compliance?
- Which control families have the largest deficiencies?
- Show me AC-2 failures.

Executive questions

- What are our five biggest risks today?
- What has changed over the past seven days?
- What should I prioritize this week?

Cost questions

- Which AI agent consumed the most tokens this month?
- What is the cost per vulnerability investigation?
- Which model provides the best cost-to-accuracy ratio?

Mandatory response format

For relevant responses, the chatbot should provide:

1. Executive summary.
2. Supporting facts.
3. Business impact.
4. Recommended action.
5. Source references.
6. Last updated timestamp.
7. Data completeness or confidence information.

If information is unavailable, it must explicitly say so.

Module 8: AI Operations and Cost Intelligence

This module monitors the AI platform itself.

Metrics

Metric	Granularity
Input tokens	Model / agent / execution
Output tokens	Model / agent / execution
Total tokens	Enterprise / agent
Model inference cost	Provider / model
Agent execution cost	Agent / workflow
Average latency	Agent / model
Success rate	Agent

Metric	Granularity
Failure rate	Agent
Retry rate	Agent
Tool-call accuracy	Agent
Human intervention	Workflow
Prompt cache utilization	Model / agent
Cost per task	Use case

Token usage must be captured from actual provider usage metadata whenever available, not estimated from character counts.

For local models, record token counts using the model's tokenizer.

For every execution, capture:

- Execution ID.
- Agent ID.
- Model ID.
- Provider.
- Input tokens.
- Output tokens.
- Cached tokens, where available.
- Inference duration.
- Estimated or actual cost.
- Retry count.
- Execution result.
- Associated use case.

Never store sensitive prompts or outputs in cost telemetry by default. Use identifiers, metadata, and approved redacted records.

PART 2. TECHNICAL ARCHITECTURE

5. Recommended technology stack

These are proposed technologies, not yet mandatory selections.

Layer	Recommended technology
Frontend	React + TypeScript
UI library	Material UI
Charts	Apache ECharts
Backend API	Python + FastAPI
Agent orchestration	LangGraph evaluation
Alternative framework	Google ADK evaluation
LLM gateway	Provider-independent abstraction

Layer	Recommended technology
Database	PostgreSQL
Event streaming	Kafka-compatible broker
Raw evidence	On-premises S3-compatible storage
Search	OpenSearch, if required
Containers	Docker
Orchestration	Kubernetes
MCP	Python MCP SDK
Authentication	Enterprise SSO / OIDC
Secrets	Enterprise secrets manager
Observability	OpenTelemetry + Prometheus/Grafana

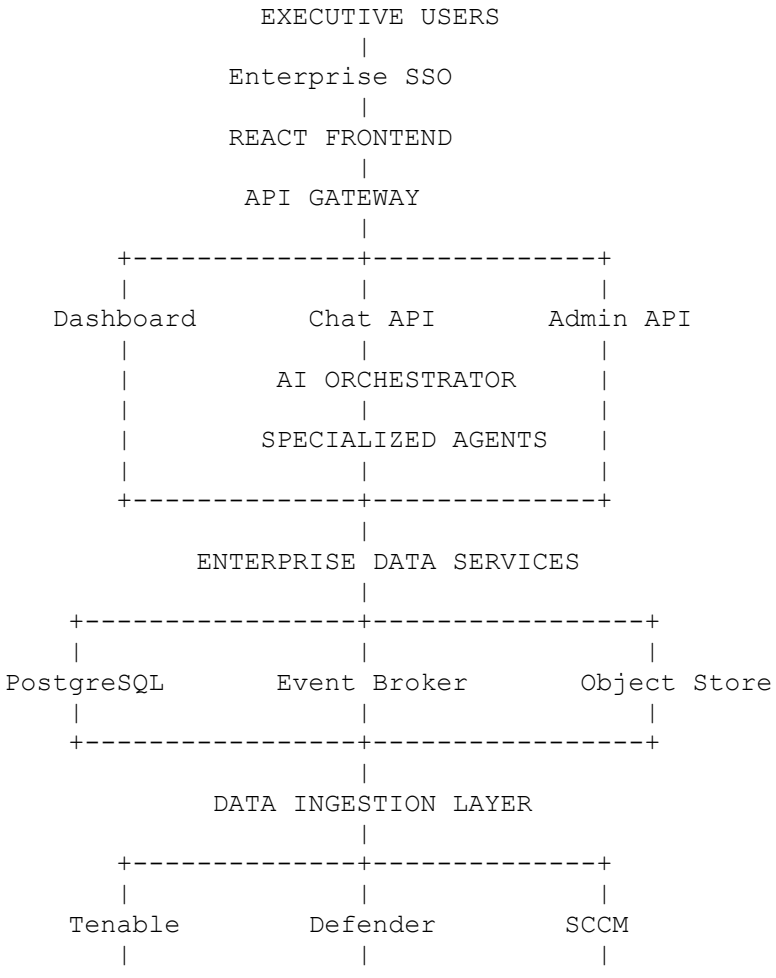
I recommend a short technical proof of concept comparing LangGraph and Google ADK before selecting the production framework.

Do not let the agent framework dictate the entire application architecture.

6. End-to-end architecture

Here is the proposed logical architecture.

The production design should have the following components.



```
Entra ID       Asset DB       Other Tools
```

Deployment principle

The data platform and security information remain on-premises.

GCP may host approved model inference or selected compute workloads only after data classification, privacy, contractual and security reviews.

No direct connection from a public LLM to the enterprise database should be permitted.

---

7. Database design

I recommend PostgreSQL as the initial operational database.

The following tables should form the foundation.

Table	Purpose
assets	Enterprise asset inventory
asset_identifiers	External identifiers and correlations
business_services	Business service catalog
applications	Application inventory
asset_relationships	Asset dependencies
vulnerabilities	Vulnerability records
asset_vulnerabilities	Asset-to-vulnerability mapping
patch_status	Patch compliance
security_controls	Control library
framework_mappings	NIST/ISO/TISAX mappings
control_assessments	Assessment results
evidence_records	Evidence metadata and provenance
risks	Enterprise risk register
risk_findings	Findings associated with risks
incidents	Security incidents
remediation_actions	Remediation ownership and status
data_sources	Integration configuration
ingestion_runs	Data collection history
agent_definitions	Agent configuration
agent_executions	Agent execution history
model_usage	Token and inference consumption
audit_events	Security and administrative audit records

Important data design requirement

Every security finding should retain:

- Original source system.
- Source record identifier.
- Collection timestamp.
- Last updated timestamp.
- Data quality status.
- Correlation status.
- Provenance.

Do not overwrite source information with AI-generated interpretations.

Keep the source facts and derived intelligence separate.

---

8. Backend API specification

The frontend and agents should communicate through documented APIs.

Initial endpoint design:

API	Method	Purpose
`/api/v1/dashboard/overview`	GET	Executive overview
`/api/v1/dashboard/trends`	GET	Historical trends
`/api/v1/assets`	GET	Asset inventory
`/api/v1/assets/{id}`	GET	Asset details
`/api/v1/assets/{id}/risk`	GET	Asset risk
`/api/v1/vulnerabilities`	GET	Vulnerability findings
`/api/v1/patch/compliance`	GET	Patch metrics
`/api/v1/controls`	GET	Control inventory
`/api/v1/controls/{id}`	GET	Control details
`/api/v1/compliance/summary`	GET	Framework compliance
`/api/v1/risks`	GET	Enterprise risk register
`/api/v1/chat`	POST	Conversational AI
`/api/v1/ai/usage`	GET	Token and cost metrics
`/api/v1/platform/health`	GET	Platform health

All APIs must implement authorization, pagination, filtering, validation, and audit logging as appropriate.

---

9. Agent engineering specifications

Every agent should follow a standardized contract.

For example:

Vulnerability Intelligence Agent

Purpose: Analyze vulnerability information and produce risk-prioritized remediation intelligence.

Inputs:

- Normalized vulnerability records.
- Asset criticality.
- Exploitability information.
- Business service relationships.
- Existing compensating controls.

Processing:

1. Validate vulnerability information.
2. Deduplicate findings.
3. Identify affected assets.
4. Evaluate exploitability.
5. Determine remediation SLA.
6. Assess business criticality.
7. Generate risk recommendations.

Outputs:

```
{
  "asset_id": "AST-10025",
  "vulnerability_id": "CVE-EXAMPLE",
  "severity": "CRITICAL",
  "business_criticality": "HIGH",
  "sla_status": "OVERDUE",
  "risk_priority": "CRITICAL",
  "recommended_action": "Prioritize remediation",
  "evidence_references": [
    "TEN-10001"
  ],
  "assessment_timestamp": "2026-11-08T12:00:00Z"
}
```

This is an illustrative schema.

Agent execution principles

All agents must:

- Have defined responsibilities.
- Use approved tools.
- Have dedicated identities.
- Follow least privilege.
- Produce structured outputs.
- Maintain execution logs.
- Report errors.
- Support retries safely.
- Track token usage.
- Support evaluation.

- Never silently convert missing data into compliant results.

Use deterministic Python logic for calculations and policy checks. Use LLMs for interpretation, contextual analysis and summarization where they add measurable value.

PART 3. UI/UX ENGINEERING SPECIFICATION

10. Main application navigation

I recommend the following navigation structure.

Navigation	Subsections
Executive Overview	Enterprise posture, top risks, trends
Asset Intelligence	IT, cloud, applications, OT placeholder
Vulnerability Management	Exposure, aging, remediation
Patch Compliance	OS, applications, SLA
Control Compliance	NIST, ISO, TISAX
Security Operations	Incidents, detection, response
Identity Security	Accounts, privileges, access
Cloud Security	GCP posture and exposure
Network Security	Firewall and segmentation
Data Protection	DLP and sensitive data
Resilience	Backup, recovery, continuity
Risk Management	Enterprise risk register
AI Operations	Agents, tokens, costs, execution
Platform Health	Integrations, data quality, availability
Administration	Users, permissions, configuration

11. Screen design requirements

I recommend developing these screens first:

Screen	Priority
Executive Overview	P0
Asset Inventory	P0
Asset Detail / Risk	P0
Vulnerability Dashboard	P0
Patch Compliance	P0
NIST Compliance Dashboard	P0
Control Detail / Evidence	P0

Screen	Priority
AI Chat Assistant	P0
AI Operations	P0
Platform Health	P0

Asset detail screen

When the CIO or CISO selects an asset, they should see:

Asset profile

- Name.
- Asset ID.
- Owner.
- Business unit.
- Criticality.
- Environment.
- Business service.

Security posture

- Vulnerabilities.
- Patch status.
- Control implementation.
- Identity exposure.
- Configuration gaps.

Risk intelligence

- Overall risk.
- Business impact.
- Top contributing factors.
- Recommended actions.

Evidence

- Source platform.
- Finding identifier.
- Last collection time.
- Assessment history.

This screen is especially important because it connects executive-level risk to actual technical evidence.

PART 4. SECURITY AND GOVERNANCE REQUIREMENTS

12. Security controls

These are mandatory design requirements.

Domain	Requirement
Authentication	Enterprise SSO
Authorization	RBAC + ABAC
Agent identity	Dedicated service/workload identities
Secrets	Centralized secrets management
MCP	Authenticated, authorized and audited tool access
API	Authentication, authorization, validation and rate limits
Data	Encryption and classification
Network	Segmentation and controlled egress
LLM	Prompt injection protection
Tool calls	Input/output validation
Evidence	Provenance and integrity
Audit	Tamper-evident audit records
Human oversight	Approval for sensitive actions

Read-only MVP

The initial platform will have no autonomous remediation permissions.

The chatbot may recommend actions but cannot:

- Disable user accounts.
- Modify firewall rules.
- Patch servers.
- Change security configurations.
- Delete assets.
- Close security findings.
- Modify control assessments without authorized workflow.

Jira ticket creation can be introduced in Phase 2 with explicit approval.

PART 5. DEVELOPMENT ROADMAP

13. Recommended implementation sequence

I would divide development into six engineering sprints.

These are proposed sprint boundaries, not committed calendar durations.

Sprint	Engineering deliverable
Sprint 0	Architecture foundation, repository, CI/CD, security baseline
Sprint 1	Enterprise Asset Intelligence and PostgreSQL data model
Sprint 2	Tenable, Defender, SCCM and Intune integrations
Sprint 3	Vulnerability and Patch Compliance intelligence
Sprint 4	NIST control engine and evidence management
Sprint 5	Executive dashboard, AI chatbot and AI operations

The sequence can be adjusted after technical validation.

MVP acceptance criteria

The MVP is ready for pilot when:

1. The existing asset database is successfully ingested.
2. External asset records can be correlated.
3. Vulnerability data is available against enterprise assets.
4. Patch compliance is calculated with unknown and exempt states handled correctly.
5. NIST control assessments can be created and supported with evidence.
6. The dashboard presents enterprise-level KPIs.
7. Users can drill down into asset and control details.
8. The AI assistant can answer questions using verified data.
9. Answers contain source references and timestamps.
10. The platform tracks token consumption and agent execution costs.
11. Platform health identifies failed or stale integrations.
12. Access controls prevent unauthorized users from accessing restricted information.

PART 6. ENGINEERING TEAM HANDOFF

This is how I would divide responsibilities.

Role	Primary responsibility
Principal AI Architect	Agent architecture and orchestration
AI Engineers	Agent implementation and evaluation
Backend Engineers	APIs, ingestion and data services
Data Engineer	Data model, normalization and correlation
Security Architect	Platform security and threat modeling
Infrastructure Architect	On-premises/GCP deployment
DevSecOps Engineer	CI/CD, containers and deployment security
UI/UX Designer	Executive experience and wireframes
Frontend Engineer	React dashboard and chatbot
Product Owner	Functional validation and prioritization

Claude Code development strategy

I recommend creating one master project repository with clearly separated modules.

```
cyber-command-center/
│
├── frontend/
│   ├── dashboard/
│   ├── asset-intelligence/
│   ├── compliance/
│   ├── vulnerability/
│   └── ai-chat/
│
├── backend/
│   ├── api/
│   ├── authentication/
│   ├── asset-service/
│   ├── vulnerability-service/
│   ├── compliance-service/
│   ├── risk-service/
│   └── ingestion/
│
├── agents/
│   ├── asset-agent/
│   ├── vulnerability-agent/
│   ├── patch-agent/
│   ├── compliance-agent/
│   ├── risk-agent/
│   └── executive-agent/
│
├── integrations/
│   ├── tenable/
│   ├── defender/
│   ├── sccm/
│   ├── intune/
│   ├── chronicle/
│   ├── entra-id/
│   └── asset-database/
│
├── mcp/
│   ├── asset-mcp/
│   ├── vulnerability-mcp/
│   ├── compliance-mcp/
│   └── jira-mcp/
│
├── data/
│   ├── models/
│   ├── migrations/
│   └── schemas/
│
├── platform/
│   ├── docker/
│   ├── kubernetes/
│   ├── monitoring/
│   └── security/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── agent-evaluation/
```

```
│   └── security/
│
└── docs/
    ├── architecture/
    ├── api/
    ├── agent-specifications/
    └── product-requirements/
```

This is the proposed logical structure. We should validate repository boundaries and deployment ownership before implementation.

PART 7. MY THREE MOST IMPORTANT RECOMMENDATIONS

Before starting development, I would insist on three engineering principles.

1. Establish data trust before AI intelligence

The platform should never generate executive risk intelligence from unreliable or stale source data.

Every metric should have provenance, collection time and completeness status.

2. Separate AI intelligence from business logic

For example:

- Python calculates patch compliance.
- Rules evaluate remediation SLAs.
- A control engine evaluates defined assessment criteria.
- AI interprets evidence and generates contextual recommendations.
- Humans approve sensitive compliance decisions.

This improves accuracy and controls cost.

3. Treat the AI platform as an enterprise product itself

Monitor:

- Agent health.
- Model health.
- Token consumption.
- Inference costs.
- Tool-call accuracy.
- Hallucination rates.
- Execution latency.
- Data quality.
- Security events.

The platform must be able to demonstrate that its own intelligence is trustworthy.

---

14. What I recommend building first

I would now move into implementation preparation.

The first engineering package should contain:

Artifact	Purpose
A. Detailed PostgreSQL schema	Establish the common security data model
B. API specifications	Define frontend/backend contracts
C. Asset Intelligence Agent specification	Define the first production agent
D. Data ingestion specifications	Define mock and production connectors
E. Executive dashboard wireframes	Give the UI developer a visual target
F. Claude Code master prompts	Generate the implementation in controlled stages
G. Agent evaluation framework	Establish accuracy and reliability benchmarks

My preferred starting point is Enterprise Asset Intelligence.

It is the foundation for vulnerability, patch, compliance and risk correlation.

I would build it using mock connectors initially, validate the complete end-to-end workflow, and then replace the mocks with authenticated integrations to your actual enterprise tools.

The OT data model should also exist from the beginning, even though OT integrations will come later.

Recommended next milestone

Let's move to **Engineering Package 1: Enterprise Asset Intelligence Foundation**.

I can develop the detailed PostgreSQL schema, API contracts, agent specifications, dashboard wireframes, test cases and executable Claude Code prompts so your engineers can begin implementation in a controlled sequence.

The first working demonstration should allow the CIO/CISO to select an asset, understand its business criticality, see its vulnerabilities and patch status, inspect its control compliance, and ask the AI assistant questions about that asset.

That will be our first meaningful end-to-end product milestone.